



A Train Station

Computer Graphics & Animation Lab (CSE 444)

Project Final Report

Section: 7

Group No.: 09

Assigned Project No.: 05

Submitted By:	
Azizul Hakim Fayaz	2023100010209
Jakaria Kabir Provat	2023100010203
Shourov Sarkar	2023100010093

1. Project Requirements

The primary goal of this project is to render an interactive 3D train station scene centered on a passenger train, station platform, tracks, and waiting facilities using Three.js. The project must fulfil the following general requirements: 3D model loading and texture mapping, lighting, perspective projection, realistic shadows, multi-stage animation, and mouse and keyboard interaction.

Project-specific requirements:

- A detailed 3D train station with platform, waiting building, track sleepers, passenger benches, and overhead canopies.
- A passenger train consisting of locomotive engine and passenger wagons with realistic textures.
- The camera can move around the train station using both mouse orbiting and keyboard navigation.
- Train headlight and cabin lights toggle ON and OFF at runtime via mouse click with visual emissive feedback.
- Multi-stage train motion animation: the train arrives, decelerates smoothly to halt at the platform for boarding, and departs along the tracks.
- Real-time on-screen HUD displaying status indicators and interactive controls.

2. Software Platform

Tool / Technology	Purpose
Three.js (v0.186)	3D rendering library built on WebGL
JavaScript (ES modules)	Scene logic, animation, keyboard/mouse interaction and event handling
GLTFLoader	Loading high-fidelity 3D glTF/GLB models (Station.glb, Engine.glb, wagon.glb)
OrbitControls	Interactive camera orbiting, damping, rotation, and distance clamping
HTML5 / CSS3	HUD overlay, status badges, instructions card, and responsive canvas layout
Vite (v8.3)	Development server and high-speed production module bundler
Netlify	Continuous deployment (CI/CD) and cloud web hosting
Node.js / npm	Package management and build scripts
Visual Studio Code	Source code editor and development environment

3. Project Features

3.1 Models and Texture Mapping

All 3D models are structured into optimized glTF/GLB assets and loaded asynchronously into the Three.js scene graph. The station environment contains intricate meshes for the station shelter, roofing, corrugated iron sheets, platforms, tracks, waste bins, platform lights, and benches. Every object incorporates physically-based materials (PBR) with diffuse, roughness, metalness, and emissive properties.

Object	Geometry	Texture and reason
Train Locomotive	Aerodynamic cab shell, bogies, front buffers, cowcatcher, dual headlights	PBR metallic blue and yellow paint textures, emissive headlight glass, glossy window panels.
Passenger Wagons	Articulated carriages, windows, bogies, suspension springs, and doors	Deep blue coach livery texture, glass windows, industrial steel roof texture.

Object	Geometry	Texture and reason
Station Platform & Canopy	Reinforced concrete platforms, textured tiles, corrugated roof panels	Weathered corrugated metal texture with rust accents, concrete slab diffuse map with yellow safety edge lines.
Railway Tracks	Steel parallel rails, wooden sleepers/ties, ballast gravel base	Ballast stone texture and rustic weathered steel rails for realistic physical track representation.
Platform Amenities	Benches, trash receptacles, informational boards, light poles	Powder-coated metal textures, glass panels, and plastic materials for authentic depot environment.

3.2 Key Interaction

Keyboard events are handled with keydown and keyup listeners tracking active keys in a state map. The camera orbits around a central focus point on the station. The arrow keys (or WASD keys) dynamically modify the spherical coordinates (theta for azimuth angle, phi for polar angle, and radius for zoom distance) relative to the target vector. The updated camera position vector is calculated in real time using Three.js `Spherical.setFromVector3()` and `setFromSpherical()`. Left/Right rotate the camera horizontally around the station, Up/Down zoom along the view ray, and Q/E adjust the elevation angle. The polar angle is clamped so the camera never clips through the ground. Key 'R' resets the camera position and focus target to the initial isometric overview.

Key	Action
Left / Right Arrow (or A / D)	Rotate / orbit camera horizontally around the station
Up / Down Arrow (or W / S)	Move camera closer or farther (zoom in / zoom out along view ray)
Q / E	Tilt camera elevation up or down (pitch angle)
R	Reset camera view and target back to the initial isometric overview

3.3 Mouse Interaction

- Left-button drag: Rotates and orbits the camera around the station target using Three.js OrbitControls with smooth damping.
- Mouse wheel: Zooms in and out smoothly within a bounded distance range (min: 5, max: 250 units).
- Click on canvas: Toggles the train's headlights and cabin lights ON and OFF. A click is separated from a drag operation by checking that the pointer moved less than 6 pixels between pointerdown and pointerup, preventing accidental light toggles during scene rotation.

3.4 Animation

- Cyclic Train Movement: The train's motion is driven by an elapsed cycle clock (total loop: 17.5s) divided into four smooth phases:
 1. Arrive Phase (6.0s): Train enters the station along the tracks, applying smooth sinusoidal ease-out deceleration ($\text{Math.sin}(\text{progress} * \text{PI} / 2)$) until coming to a complete stop at the platform.
 2. Dwell Phase (4.5s): Train remains stationary at the passenger platform (position.z = 0) allowing passenger boarding, while HUD displays 'Train: At Platform (Stopped)'.
 3. Depart Phase (5.5s): Train smoothly accelerates away along the track out of the station using sinusoidal ease-in acceleration ($1 - \text{Math.cos}(\text{progress} * \text{PI} / 2)$).
 4. Away Phase (1.5s): Train resets position along the approach track outside the station.
- Camera Motion Damping: Camera movement and rotation are smoothly interpolated each frame with damping for fluid, cinematic navigation.
- RequestAnimationFrame Loop: All animations execute inside a requestAnimationFrame render loop using Three.js Clock.getDelta(), ensuring uniform speed across different monitor refresh rates.

3.5 Lighting

Three lighting types are combined, and the Three.js MeshStandardMaterial uses a physically-based (PBR) lighting model where surface rendering depends on normals, lighting angles, roughness, and metalness:

- Directional Light (Sun): Warm directional sunlight (0xfff5e6, intensity: 2.0) positioned at (45, 90, 50) casting realistic shadows across station roofs, platforms, and tracks using a 2048x2048 shadow map.
- Hemisphere Light: Ambient sky-to-ground light (0xddeeff sky, 0x161720 ground, intensity: 1.4) that illuminates shadowed areas and prevents black pitch-black voids.
- Fill Directional Light: Cool secondary directional light (0x7090b8, intensity: 0.8) positioned at (-50, 40, -40) providing complementary architectural depth.
- Emissive Train Headlights: Dynamic emissive material (Light_Texture) with intensity 6.0 creating vibrant train headlights.

Shadows are enabled with PCFSoftShadowMap for realistic soft edges, and ACESFilmicToneMapping ensures bright areas avoid harsh over-exposure.

3.6 Custom Shaders and Material Control

Scene graph traversal automatically locates all light meshes and Light_Texture materials, caching their original emissive intensities. When toggled, the materials dynamically switch RGB values and emissive intensities between active and inactive states.

- **Material Uniforms & State:** The `Light_Texture` material uses dynamic emissive properties. When activated, emissive intensity is boosted to 6.0 with pure white RGB (1, 1, 1). When deactivated, RGB is set to (0, 0, 0) with zero intensity, giving a realistic power-off appearance.
- **Atmospheric Depth:** Configured with `THREE.FogExp2` and dark charcoal clear color (0x111116) to seamlessly blend distant tracks and geometry into the horizon.
- **High Performance:** Configured with `powerPreference`: 'high-performance' and device pixel ratio clamped to 2.0 for smooth 60fps rendering across desktop and mobile devices.

Table 01: Project Feature Table

#	Features	Status
1	3D Train Station Scene with Platform & Architecture	Implemented
2	Realistic Passenger Train (Locomotive & Wagons)	Implemented
3	Realistic PBR Materials & High-Res Textures	Implemented
4	Directional Sunlight & Hemisphere Lighting	Implemented
5	Soft Shadow Mapping (PCFSoftShadowMap)	Implemented
6	Perspective Projection (PerspectiveCamera)	Implemented
7	Multi-Phase Train Arrival & Departure Animation	Implemented
8	Train Headlights Emissive Light Toggle	Implemented
9	Keyboard Navigation (WASD / Arrow Keys Orbit & Zoom)	Implemented
10	Mouse Interaction (OrbitControls & Click Detection)	Implemented
11	Interactive HUD with Real-time Status Badges	Implemented
12	View Reset Functionality (Key 'R')	Implemented
13	Responsive WebGL Canvas & ACES Filmic Tone Mapping	Implemented
14	Automated Netlify CI/CD Deployment	Implemented
15	Procedural Ambient Particles (Fog & Smoke)	Partially Implemented
16	Multi-Track Switching Logic	Not Implemented

Note: The camera was intentionally designed with both mouse and keyboard orbit capabilities, allowing complete interactive exploration of the train station platform, rails, and locomotive details.

4. Snapshots

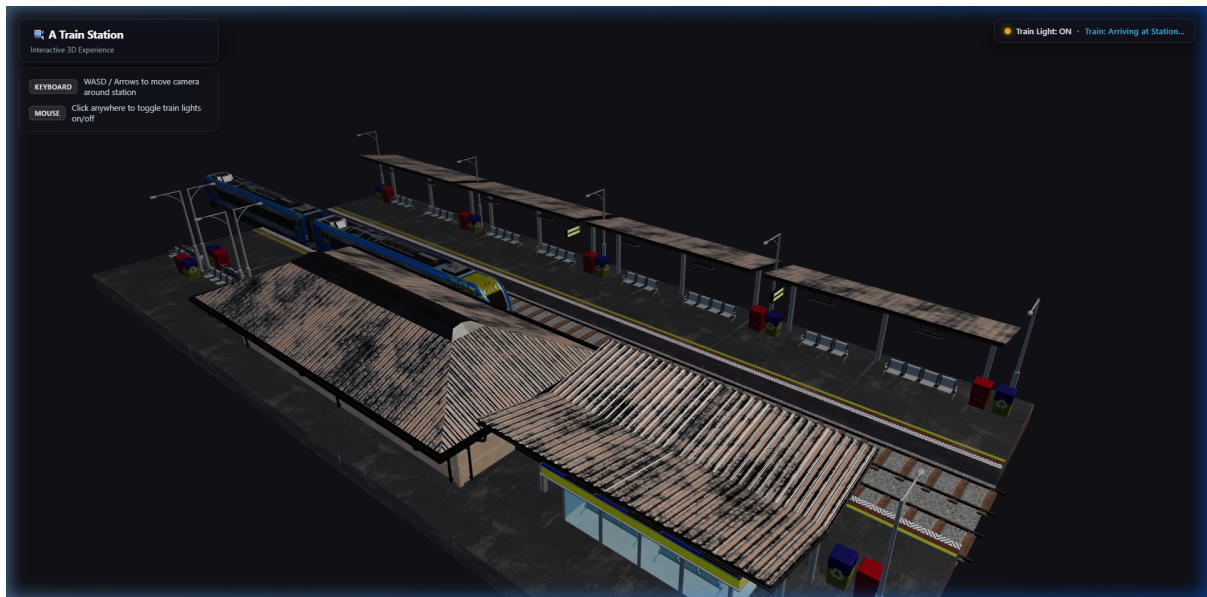


Figure 1: Full scene overview (station, tracks, train, waiting building, platform)

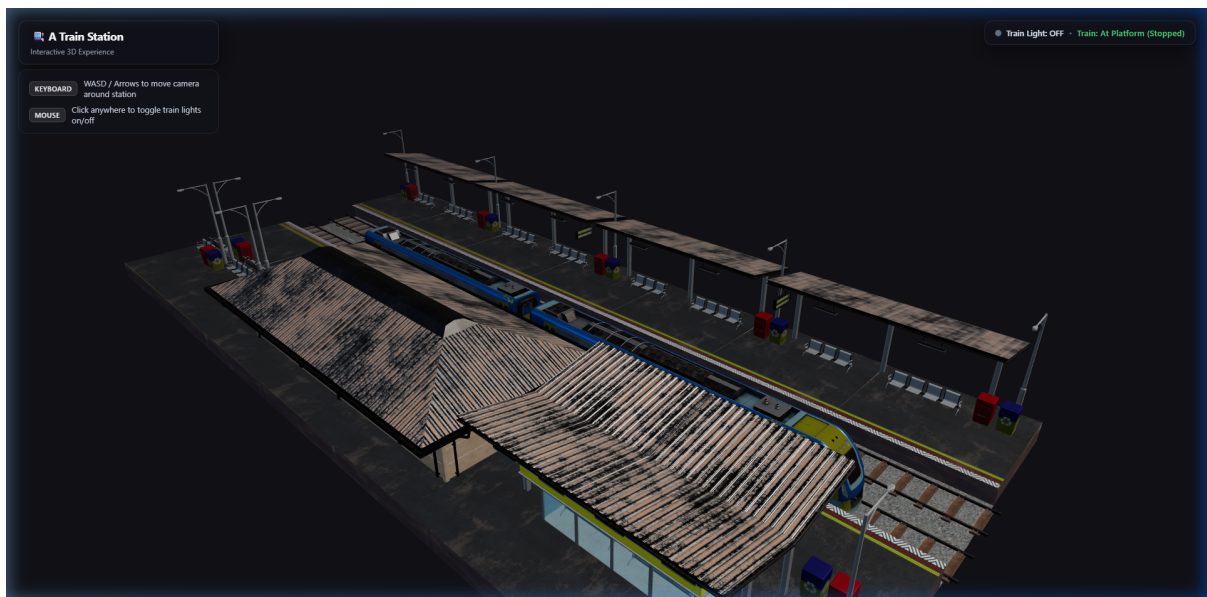


Figure 2: Train stopped at platform with passenger cars and station benches

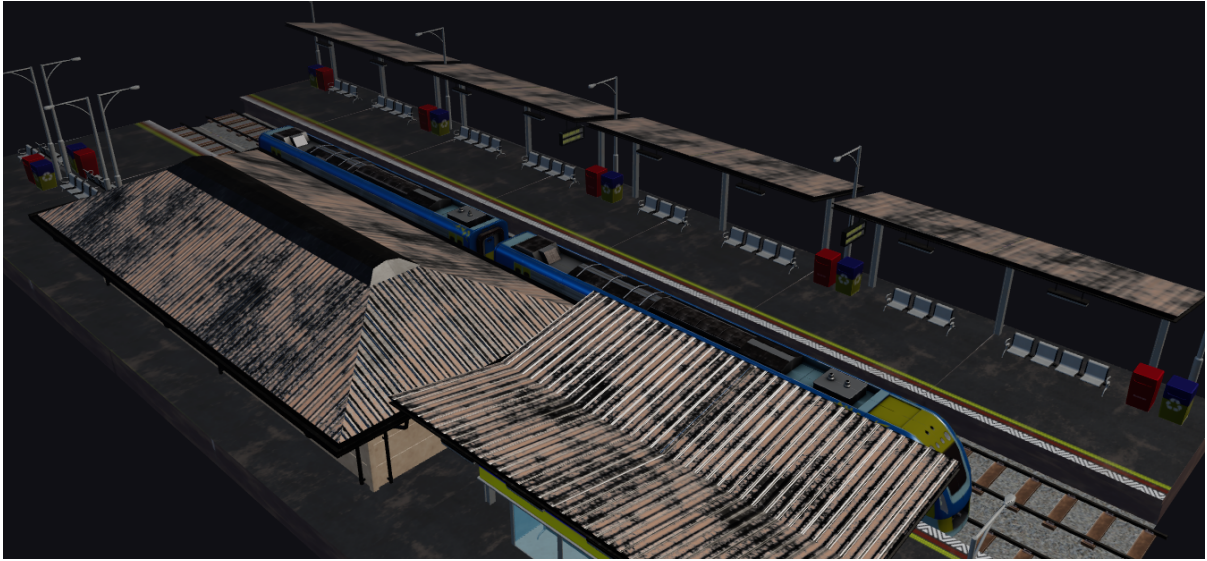


Figure 3: Close-up of passenger train arriving at the platform

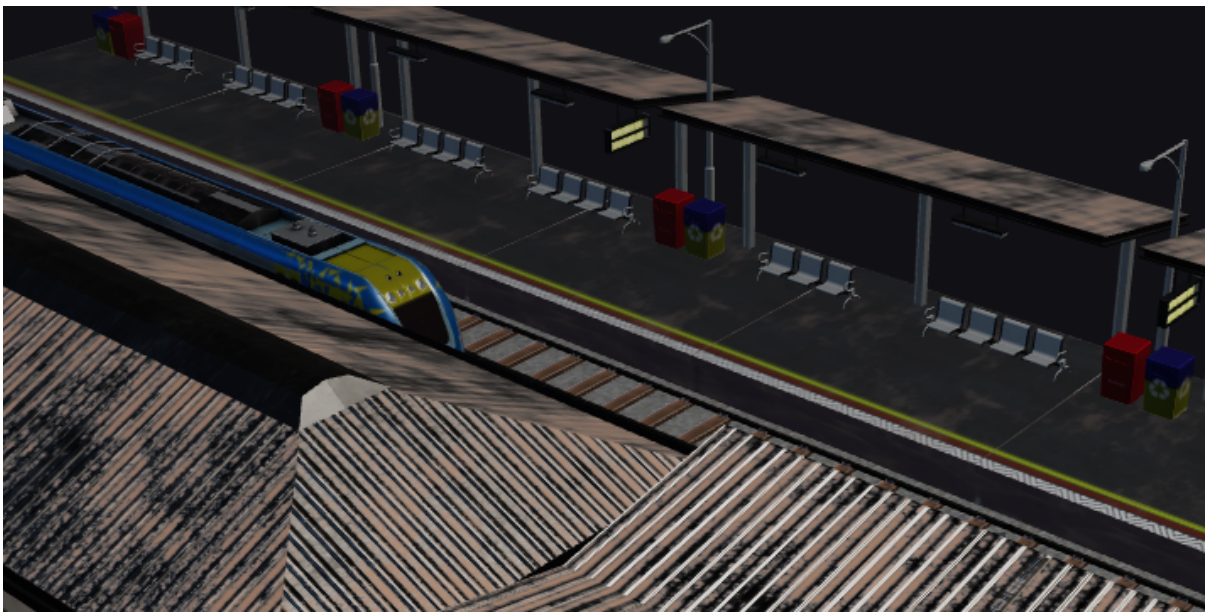


Figure 4: Train headlights and cabin illumination ON (Emissive state: ON)

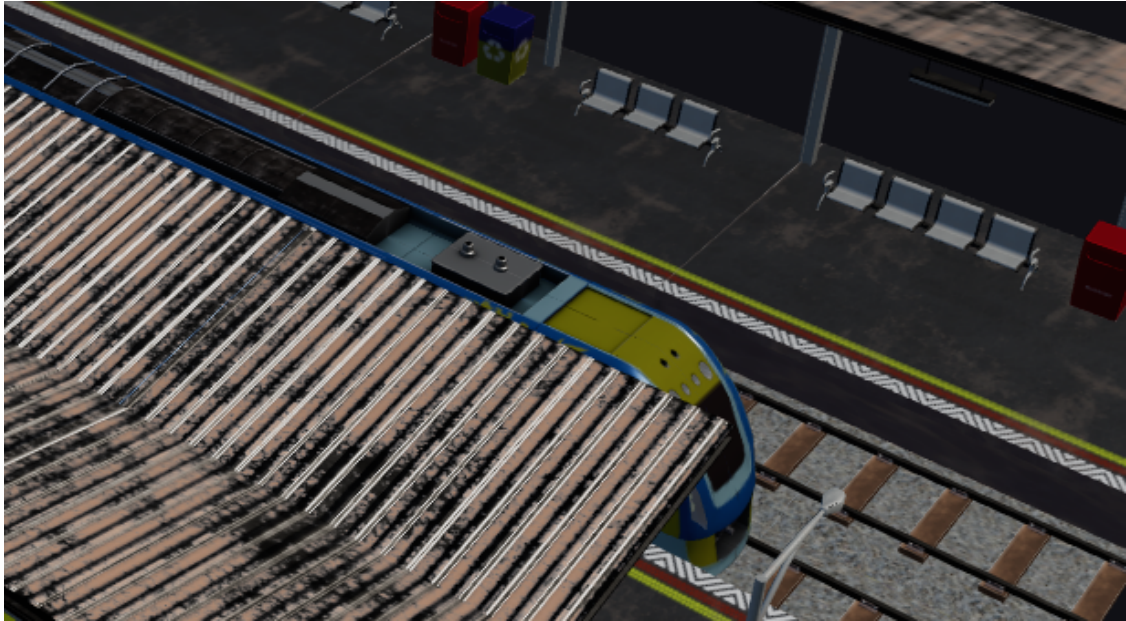


Figure 5: Train headlights toggled OFF via interactive mouse click

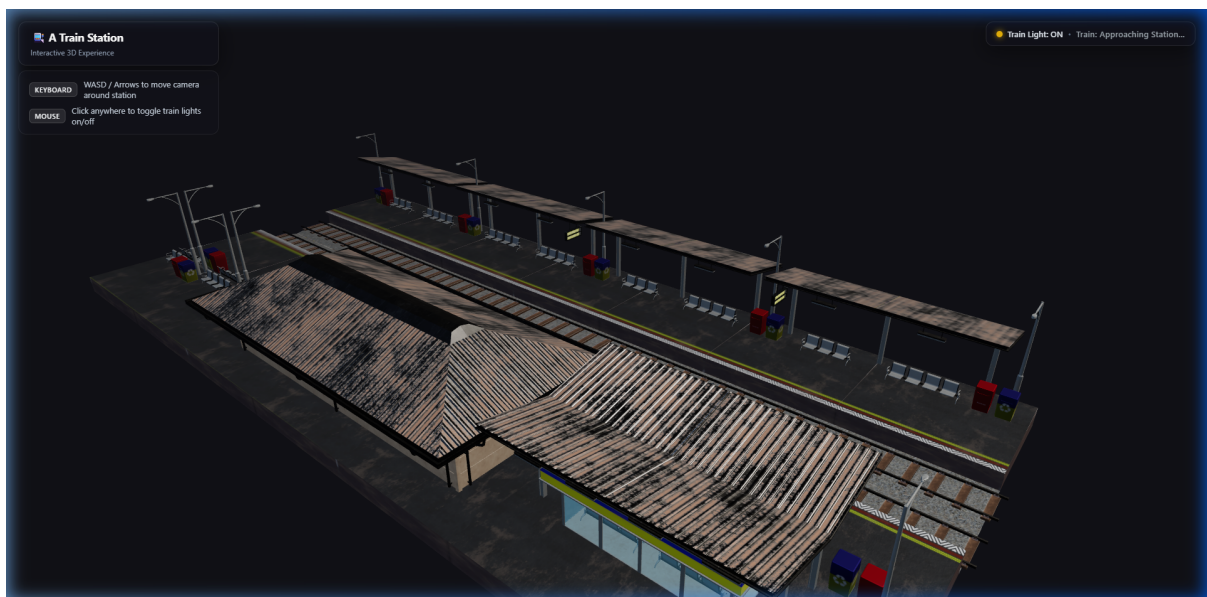


Figure 6: Detailed view of station platform canopy, track sleepers, and architectural assets

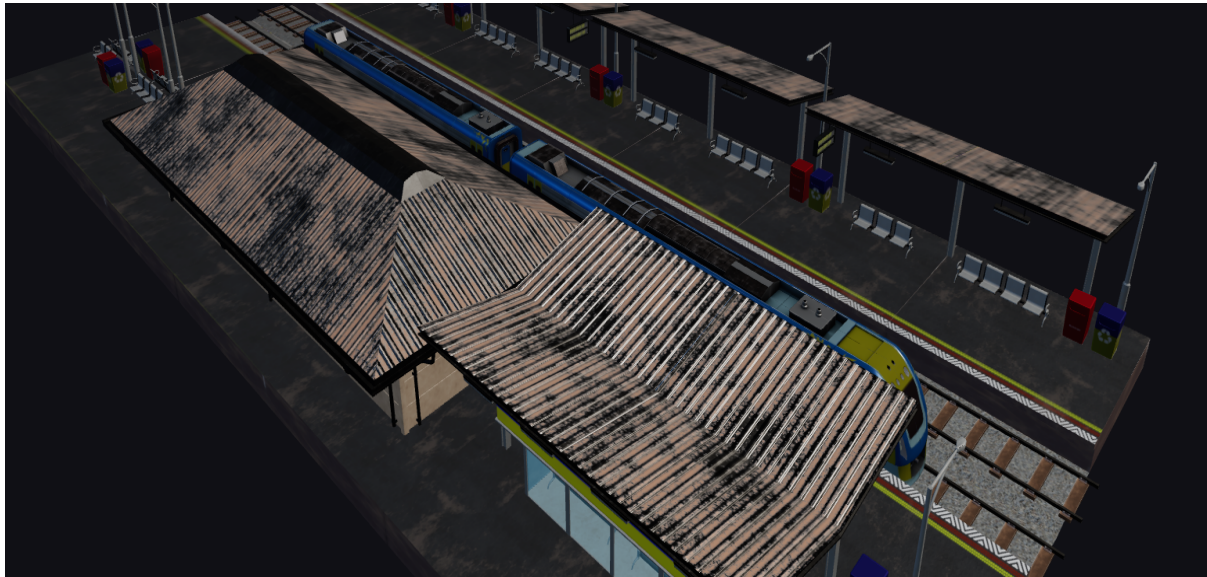


Figure 7: Camera rotated to view track layout and platform structure

Project Live URL: <https://threejs-train-station.netlify.app/>

GitHub Repository: <https://github.com/AzizulHakimFayaz/ThreeJS-Train-Station>

5. Contribution

Member	Contribution
Azizul Hakim Fayaz (2023100010209)	3D Scene setup, GLTF model integration, lighting configuration, train grouping and hierarchical animation.
Jakaria Kabir Provat (2023100010203)	Keyboard and mouse camera controls, OrbitControls integration, camera reset system, and light toggle logic.
Shourov Sarkar (2023100010093)	UI/HUD styling, status indicators, shadow mapping optimization, project report documentation, and Netlify deployment.